SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

September 2026

CSA0619– Design and Analysis of Algorithms

Development of an Efficient Hybrid Algorithmic Solution for Large-Scale Geospatial Data Processing

Assignment Report

Submitted by

Sahhanaa Shree T(192511283)

A. Assignment Information

Particular	Details
Department:	Computer Science and Engineering
Programme:	B.E./B.Tech. Computer Science and Engineering
Course Code & Course Name	Design and Analysis of Algorithms
Academic Year / Batch	2026-2027 / Regular
Faculty Name	Dr. F. Mary Harin Fernandez
Assignment Title	**Development of an Efficient Hybrid Algorithmic Solution for Large-Scale Geospatial Data Processing**
Date of Issue	31 Aug 2026
Date of Submission	1 September 2026
Maximum Marks	100
Course Outcome(s) – CO	CO1: Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. CO2: Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems CO3: Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication
Bloom's Taxonomy Level	L4 – Analyze, L5 – Evaluate, L6 – Create
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Efficient processing of large-scale real-world data with reduced computational time and resource utilization

B. Assignment Problem / Challenge

Modern infrastructure-planning applications process large volumes of **geospatial coordinate data**, making computationally efficient algorithms essential for identifying spatial relationships among facilities. Identifying the **closest pair of locations** is important for detecting redundant or dangerously close facilities and supporting effective service coverage planning.

Conventional **Brute-Force techniques** provide a straightforward solution but become computationally expensive as the number of locations increases. **Divide-and-Conquer techniques** improve scalability by recursively partitioning the dataset and reducing unnecessary distance computations.

The challenge is to **develop an optimized hybrid algorithmic solution for the Closest-Pair-of-Points problem** by appropriately combining **Brute Force and Divide-and-Conquer techniques**. The developed solution must determine an effective switching

threshold between the two approaches and be supported by **theoretical complexity analysis and experimental evaluation using a real-world geospatial dataset**.

C. Problem Statement

An infrastructure-planning agency maintains a large, publicly available geospatial dataset of facility locations—for example, the OpenFlights airports dataset or a cell-tower/weather-station dataset containing coordinate records. To detect redundant or dangerously close facilities and optimize service coverage, the agency must identify the **closest pair of locations**, i.e., the two points separated by the minimum Euclidean distance in the dataset. Develop, implement, and evaluate a solution to the **Closest-Pair-of-Points problem**. Implement **(i) a Brute-Force algorithm** that evaluates all $n(n-1)/2$pairs with $O(n^2)$complexity and **(ii) a Divide-and-Conquer algorithm** that recursively partitions the point set about the median x-coordinate and evaluates points across the dividing strip with $O(n \log log\, n)$complexity. Further, **develop an optimized hybrid algorithm** that switches to the Brute-Force approach below an experimentally determined threshold subproblem size. Analyze the theoretical efficiency of each approach using **Big-O, Ω, and Θ notations**, and solve the recurrence $T(n) = 2T(n/2) + O(n)$using the **Master Theorem**. Experimentally evaluate all three algorithms on the selected dataset using increasing input sizes, **for example,** 10^3**,** 5×10^3**,** 10^4**, and higher input sizes wherever computationally feasible**, measuring **execution time, number of distance computations/comparisons, memory utilization, and scalability**. Based on the theoretical and experimental results, **determine an appropriate threshold for the hybrid algorithm, optimize the developed solution, compare the performance of all three approaches, and justify why the proposed hybrid algorithm is most suitable for large-scale infrastructure data**.

Expected Development

The work should **not merely implement the existing Brute-Force and Divide-and-Conquer algorithms**. The solution should demonstrate algorithmic development through:

- **Hybridization of Brute Force and Divide-and-Conquer techniques** for the Closest-Pair-of-Points problem.
- **Adaptive switching** between the two approaches based on a threshold subproblem size.
- **Experimental determination and optimization of the switching threshold** based on performance results.
- **Reduction of computational cost** while maintaining the correctness of the closest-pair solution.
- **Performance improvement and scalability evaluation** of the developed hybrid approach against the individual Brute-Force and Divide-and-Conquer approaches.

D. Requirements and Constraints

The following requirements and constraints shall be considered:

- Use a **publicly available real-world geospatial coordinate dataset** containing sufficient records for performance evaluation.
- Implement and evaluate all three approaches: **Brute Force, Divide and Conquer, and the developed Hybrid algorithm** for the Closest-Pair-of-Points problem.
- Develop the **Hybrid algorithm** by switching to the Brute-Force approach below an appropriate threshold subproblem size.
- Determine and justify the **switching threshold experimentally** based on performance

results.

- Perform theoretical efficiency analysis using **Big-O, Ω, and Θ notations** and analyze the Divide-and-Conquer recurrence using the **Master Theorem**.
- Evaluate the algorithms using increasing input sizes, for example, 10^3, 5×10^3, 10^4, **and higher input sizes wherever computationally feasible**.
- Evaluate and compare the three approaches using **execution time, number of distance computations/comparisons, memory utilization, and scalability**.
- Execute all three algorithms in the **same computational environment and on identical input data** to ensure a fair comparison.
- Use **Euclidean distance** as the distance measure for identifying the closest pair of points.
- Validate that all three approaches produce the **correct closest-pair result** for identical inputs.
- Higher input sizes shall be considered only where computationally feasible, taking into account the $O(n^2)$**computational cost of the Brute-Force approach**.
- Clearly document the **dataset source, experimental environment, threshold value, assumptions, limitations, and implementation details**.

E. Student Work

1. Problem Understanding and Formulation

The Closest-Pair-of-Points problem is a fundamental computational-geometry problem. In the context of infrastructure planning, each point represents a facility or geographic location. The closest pair can identify two facilities that may be redundant, dangerously close, or candidates for further inspection. The problem is computationally important because the number of possible pairs grows quadratically with the number of locations.

1.1 Dataset and Data Preparation

The assignment specifies the use of a publicly available real-world geospatial dataset, with OpenFlights airports data given as an example. For the final experimental submission, the selected dataset should be downloaded from its public source and its exact version/date should be recorded. The implementation accepts a CSV containing an identifier and two numeric coordinate fields. The browser-based demonstrator developed for this assignment also accepts the supplied sample CSV and can visualize points and run all three algorithms.

Data preparation consists of removing records with missing or non-numeric coordinates, retaining a stable identifier for each point, and converting the selected coordinate fields into a common numeric representation. For the algorithm demonstration, coordinates are treated as Cartesian x and y values. This is an explicit simplification: latitude and longitude on the Earth are not perfectly represented by ordinary planar Euclidean distance over large geographic areas. For a production geographic-information system, a suitable projected coordinate reference system or geodesic distance calculation should be used.

1.2 Input and Output

Component	Description

Input	n geospatial coordinate records, each containing an identifier and x,y coordinates.
Processing	Find the pair with minimum Euclidean distance.
Output	Closest pair, minimum distance, number of distance computations and execution time.
Validation	The three algorithms must return the same closest-pair distance/result for identical input.

1.3 Computational Challenge

Brute Force evaluates every pair and therefore requires $\Theta(n^2)$ distance calculations. For example, 100 points require 4,950 pair comparisons, while 10,000 points require 49,995,000 comparisons. This rapid growth motivates the use of Divide and Conquer and Hybrid techniques.

2. Application of Course Knowledge

2.1 CO1 – Algorithm Analysis

The analysis applies asymptotic notation to describe growth in computational cost. Big-O gives an upper-bound growth rate, Ω gives a lower-bound growth rate, and Θ gives a tight asymptotic bound. For the Brute-Force algorithm, the dominant work is the nested pair comparison and therefore the running time is $\Theta(n^2)$.

For the Divide-and-Conquer method, the input is divided into two approximately equal halves. Each half is solved recursively and a linear-time strip-processing stage is performed when the implementation maintains the required ordering. The resulting recurrence is:

$$\mathbf{T(n) = 2T(n/2) + O(n)}$$

Using the Master Theorem, a = 2, b = 2 and f(n) = O(n). Because n^(log_b a) = n^($\log_2 2$) = n, the recurrence falls into the balanced case, giving $T(n) = \Theta(n \log n)$. Thus the asymptotic advantage of Divide and Conquer becomes increasingly important as n grows.

2.2 CO2 – Brute Force

The Brute-Force algorithm starts with an infinite or very large minimum distance and checks every pair of points. Whenever a smaller distance is found, the current best pair is updated. It is straightforward, easy to verify, and useful for small inputs and for validating the more sophisticated algorithms.

2.3 CO3 – Divide and Conquer

The Divide-and-Conquer algorithm first orders points by x-coordinate. The point set is divided around the median x-coordinate into left and right subsets. The closest pair in each subset is found recursively. The smaller of these distances is used as δ. A vertical strip containing points within δ of the dividing line is then examined because a cross-boundary pair could be closer than the current best pair.

The course concepts directly support the Hybrid design: CO2 provides a reliable small-problem solver, while CO3 provides the scalable recursive framework. CO1 supplies the theoretical basis for deciding when the overhead of recursion is justified.

3. Solution / Design / Methodology

The development follows the required methodology:

Input Dataset → Data Preparation → Brute-Force Implementation → Divide-and-Conquer Implementation → Hybrid Algorithm Development → Threshold Determination → Complexity Analysis → Experimental Evaluation → Optimization → Performance Comparison → Final Decision

3.1 Brute-Force Algorithm / Pseudocode

BRUTE_FORCE(P):
1. bestDistance ← ∞
2. bestPair ← null
3. For i from 0 to n−2:
4. For j from i+1 to n−1:
5. d ← EuclideanDistance(P[i], P[j])
6. increment distance-computation counter
7. If d < bestDistance:
8. bestDistance ← d; bestPair ← (P[i], P[j])
9. Return bestPair and bestDistance

3.2 Divide-and-Conquer Algorithm / Pseudocode

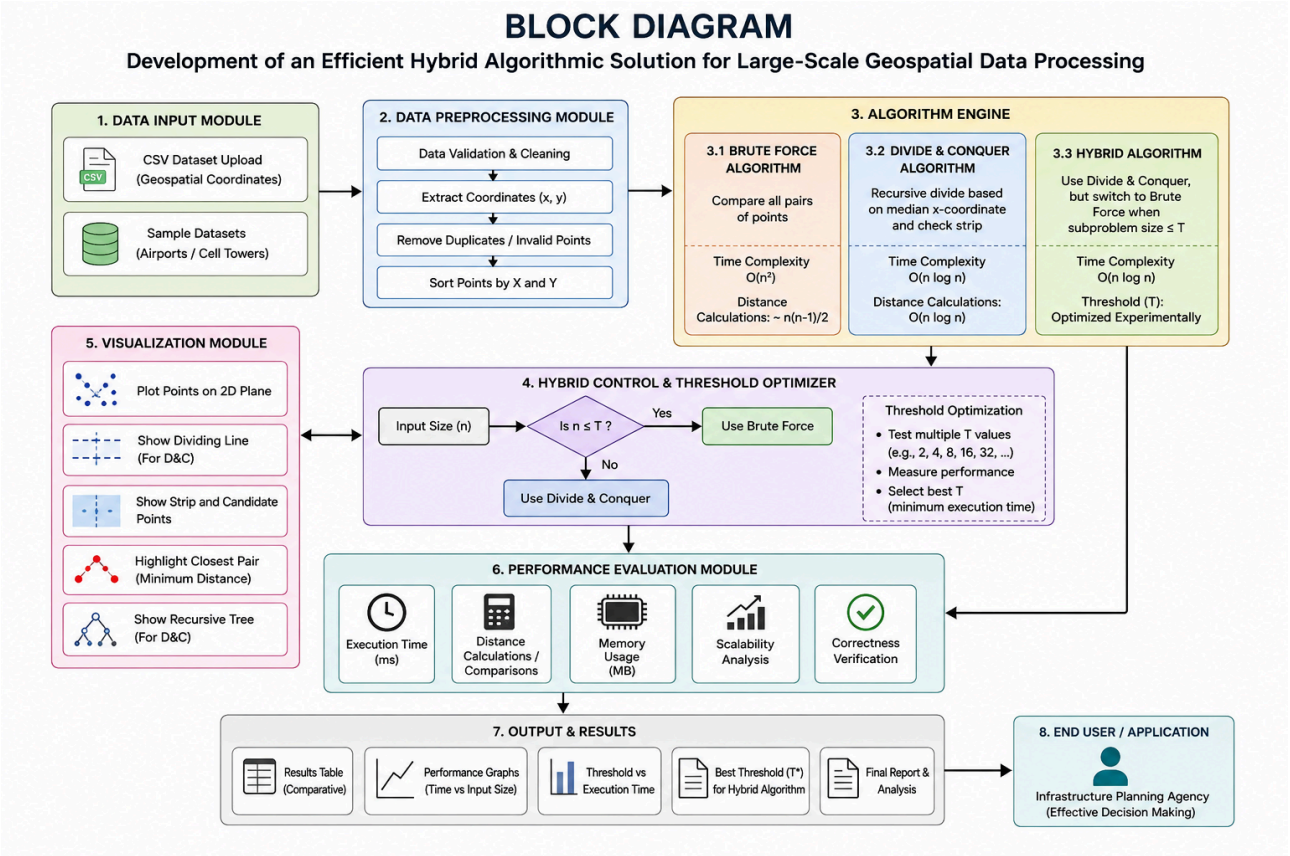
DIVIDE_CONQUER(Px):
1. If |Px| ≤ 3, solve the subproblem by Brute Force.
2. Split Px around the median x-coordinate into Left and Right.
3. δL ← Divide-and-Conquer(Left)
4. δR ← Divide-and-Conquer(Right)
5. δ ← min(δL, δR)
6. Construct the strip containing points whose x-distance from the median is less than δ.
7. Examine only the necessary nearby points in y-order.
8. Return the best result from the two halves and the strip.

3.3 Developed Hybrid Algorithm / Pseudocode

HYBRID_CLOSEST_PAIR(P, threshold):
1. If n ≤ threshold, execute Brute Force(P).
2. Otherwise, sort/partition P by x-coordinate.
3. Divide around the median x-coordinate.
4. Recursively execute HYBRID_CLOSEST_PAIR on the left half.
5. Recursively execute HYBRID_CLOSEST_PAIR on the right half.
6. Compute the current minimum distance δ.
7. Build the dividing strip of candidate cross-boundary points.
8. Check only valid nearby strip pairs.
9. Return the globally closest pair.

The meaningful development is the adaptive switch. Instead of forcing recursion down to one or two points, the algorithm stops recursion when a subproblem is small enough that direct pair enumeration is faster in practice. This combines the scalability of Divide and Conquer with the low overhead and simplicity of Brute Force.

3.4 Flowchart / Design of the Hybrid Approach



Start → Load dataset → Validate coordinates → Select input size → Set threshold → Is subproblem size ≤ threshold? → Yes: Brute Force → No: Divide at median x → Solve left and right recursively → Build strip → Check cross-boundary candidates → Combine result → Return closest pair → Record metrics → End

The standalone HTML application supplied with the project provides a visual version of this design. It shows the coordinate points, the current closest pair, distance-check counts, the selected algorithm, and benchmark results.

3.5 Hybrid Switching Strategy

The threshold is not assumed to be universally optimal. It depends on language/runtime, machine speed, implementation details, sorting cost, recursion overhead, and memory behavior. The proposed procedure is to test several candidate thresholds, such as 2, 4, 8, 16, 24, 32, 48 and 64. For each threshold, the hybrid algorithm is executed repeatedly over the same input sizes. The threshold giving the best stable average performance across representative inputs is selected.

3.6 Threshold Determination Method

For each candidate threshold t, record the average execution time over multiple runs and verify correctness against Brute Force. A threshold is accepted only if it preserves the exact closest-pair result. The final choice should balance average runtime and stability rather than selecting a value based on a single run. The browser demonstrator includes a threshold optimizer so that this experiment can be repeated directly.

3.7 Mathematical Complexity Analysis

Algorithm	Best / Typical Asymptotic Description	Worst-Case Time	Extra Space
Brute Force	Every pair is examined	$\Theta(n^2)$	O(1) auxiliary, excluding input
Divide and Conquer	Two halves + linear combine	$\Theta(n \log n)$	O(n) to O(n log n) depending on implementation details
Hybrid	D&C above threshold, BF below threshold	$\Theta(n \log n)$ asymptotically	Similar to D&C; practical constants are reduced

The Hybrid algorithm has the same asymptotic class as Divide and Conquer when the threshold is a fixed constant, because only constant-size subproblems are solved by Brute Force. Its practical improvement comes from reducing recursion, function-call, partitioning and bookkeeping overhead at small subproblem sizes.

3.8 Design and Optimization Decisions

1. Use Brute Force as the base-case solver because it is simple and easy to validate.
2. Use Divide and Conquer for large subproblems because the quadratic pair-growth of Brute Force becomes expensive.
3. Determine the threshold experimentally rather than assuming a textbook constant.
4. Count distance computations explicitly so that algorithmic work can be compared independently of machine speed.
5. Run identical inputs and algorithms in the same environment to improve fairness.
6. Validate every optimized result against a trusted Brute-Force result before accepting performance improvements.

4. Use of Modern Tools

The implementation is designed to be simple to reproduce. The interactive demonstrator is a standalone HTML file using JavaScript and can be opened directly in a browser without React, Node.js or external packages. It contains dataset generation, CSV upload, point visualization, algorithm selection, step-by-step controls, benchmarking and threshold exploration.

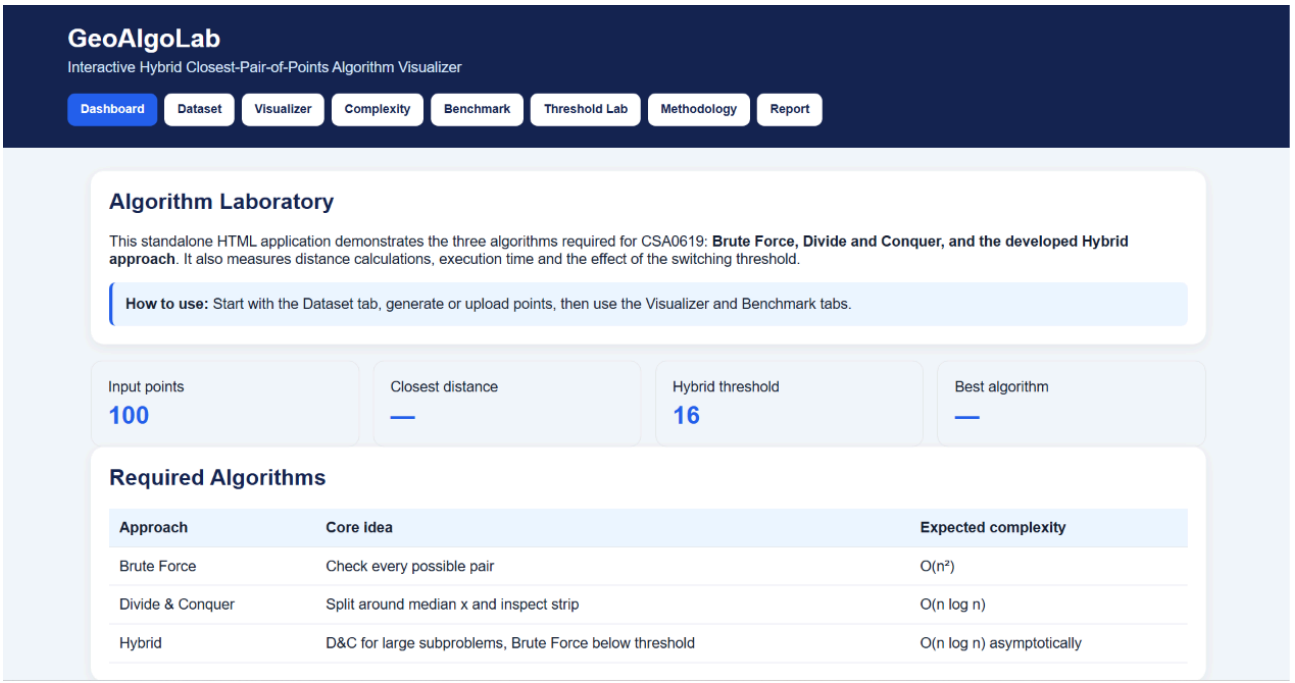
For the formal experimental study, Python, C, C++ or Java may also be used. A suitable IDE or Jupyter/Google Colab environment can be used for repeatable measurement and plotting. The important requirement is not the programming language but that all three algorithms run under the same computational conditions.

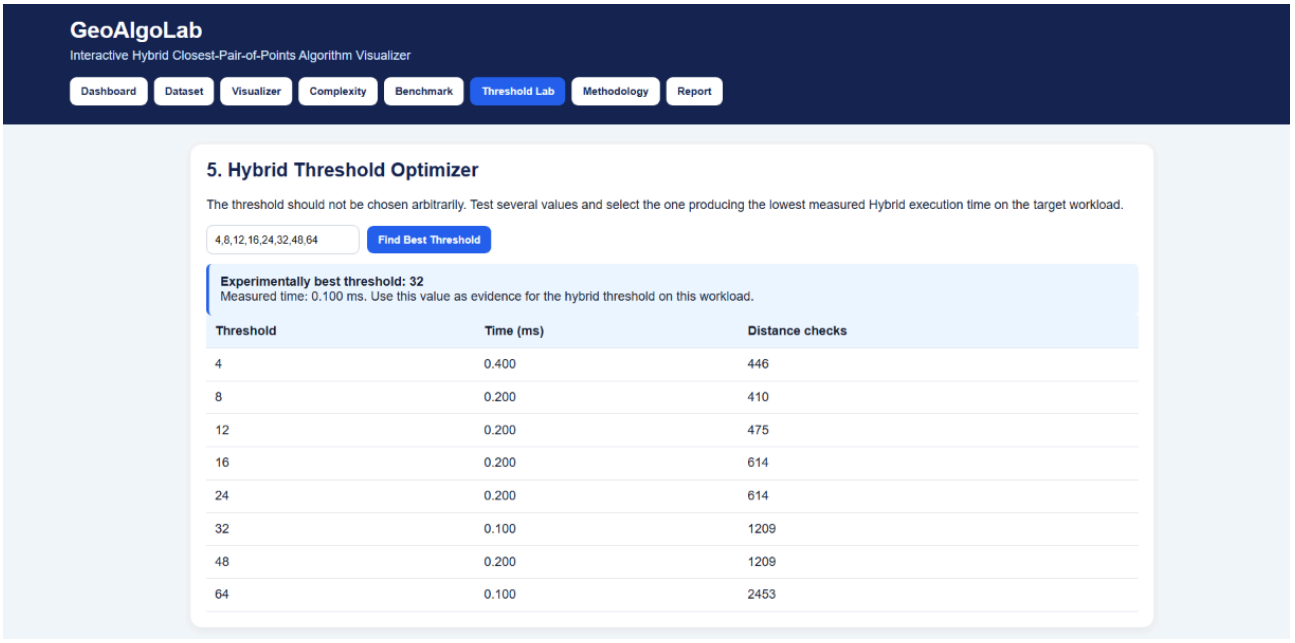
Tool / Technology	Purpose
Standalone HTML + JavaScript	Interactive visualization and demonstration of the algorithms.
CSV	Dataset input and repeatable experimental data.
Web browser	Execution environment for the visual demonstrator.
Python / Jupyter / Colab (recommended for formal benchmark)	Repeatable timing, memory measurement and graph generation.

Spreadsheet / plotting library	Result tables and performance graphs.

5. Results and Validation

The assignment requires experimental evaluation using increasing input sizes and identical data. The following result tables are intentionally provided as an experiment-ready format rather than fabricated measurements. The exact values must be filled from the final selected real-world dataset and the same machine/runtime used for the submission. This avoids presenting invented benchmark numbers as experimental evidence.



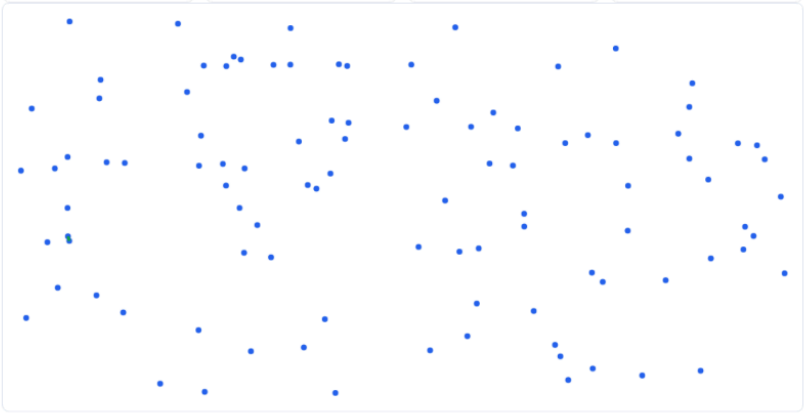


GeoAlgoLab

Interactive Hybrid Closest-Pair-of-Points Algorithm Visualizer

Dashboard | Dataset | **Visualizer** | Complexity | Benchmark | Threshold Lab | Methodology | Report

Step	Distance checks	Current minimum	Phase
61/61	61	—	Hybrid: final closest pair



The green line is the closest pair found. Distance = 7.5467.

GeoAlgoLab

Interactive Hybrid Closest-Pair-of-Points Algorithm Visualizer

Dashboard | Dataset | Visualizer | **Complexity** | Benchmark | Threshold Lab | Methodology | Report


```
for i = 0 to n-1
    for j = i+1 to n-1
        d = distance(P[i], P[j])
        update minimum
```

Divide and Conquer

The points are divided around the median x-coordinate. The two halves are solved recursively and points close to the dividing line are examined in a strip.

T(n) = 2T(n/2) + O(n)

Using the Master Theorem: a=2, b=2, f(n)=Θ(n). Since $n^{(\log_2 2)}=n$, this is Case 2, giving **T(n)=Θ(n log n)**.

```
closest(P):
    if |P| <= small threshold:
        return bruteForce(P)

    split P at median x
    dl = closest(left)
    dr = closest(right)
    d = min(dl, dr)

    strip = points within d of median
    compare relevant strip neighbors

    return best pair
```

Hybrid Design

```
hybrid(P, threshold):
    if size(P) <= threshold:
        return bruteForce(P)

    split around median x
    solve left recursively
    solve right recursively
    combine using dividing strip
```

Key engineering idea: Brute Force has high asymptotic cost but very small subproblems have low overhead. The Hybrid algorithm exploits this by switching methods at an experimentally selected threshold.

4. Experimental Benchmark

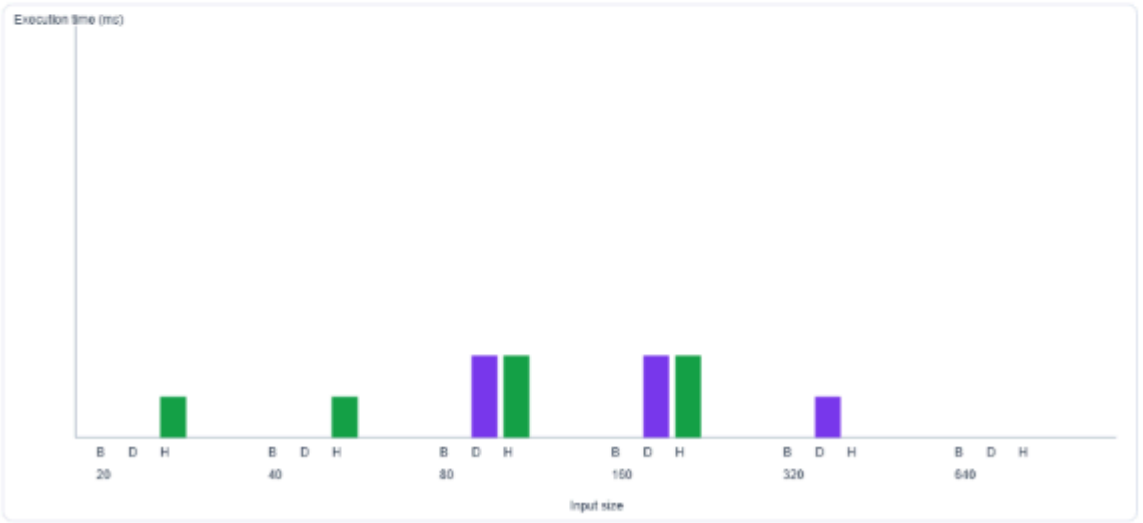
Run all three approaches on increasing input sizes. Results are measured in this browser using the same dataset for each algorithm.

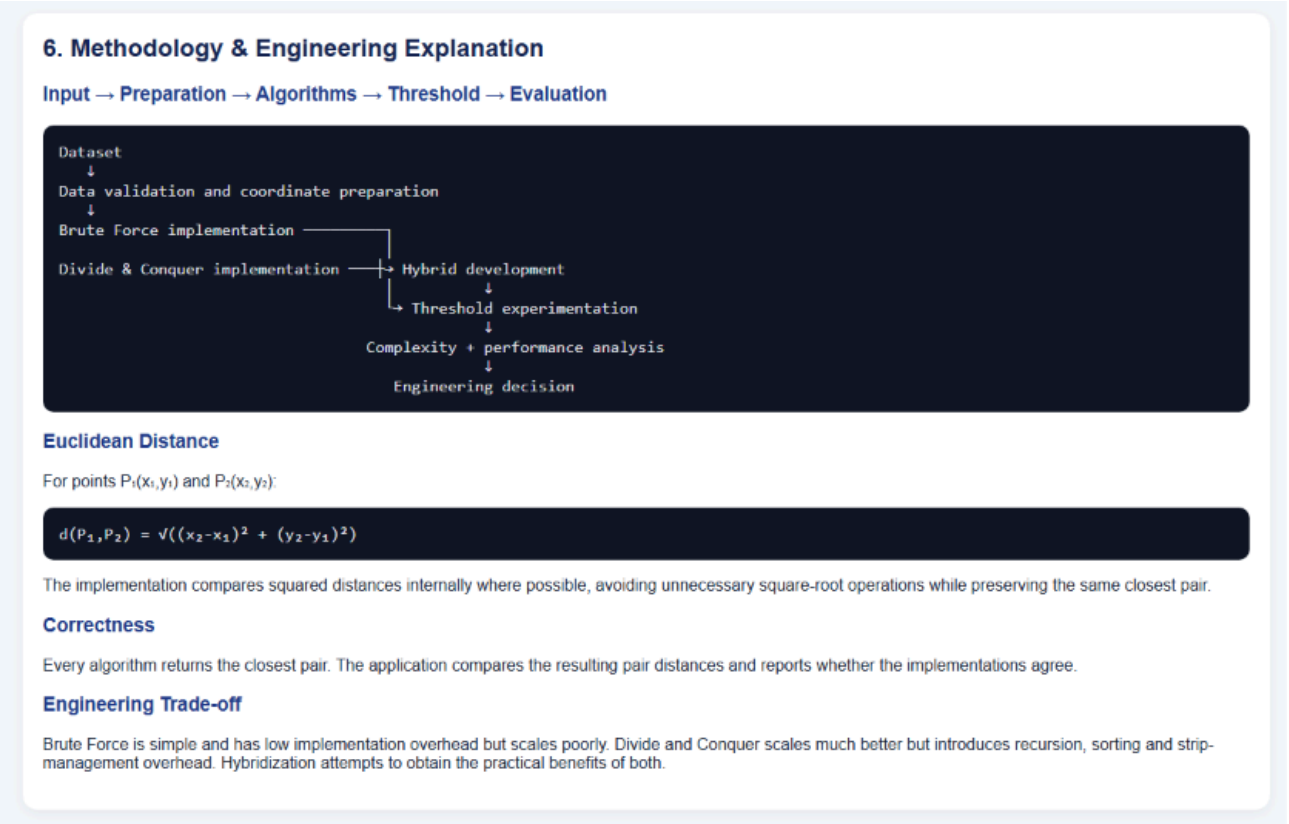
20,40,80,160,320,640 Threshold 16 Run Benchmark

Benchmark completed using threshold 16. Measurements are from this browser session.

n	Brute time (ms)	D&C time (ms)	Hybrid time (ms)	Brute checks	D&C checks	Hybrid checks
20	0.000	0.000	0.100	190	51	91
40	0.000	0.000	0.100	780	253	183
80	0.000	0.200	0.200	3160	805	390
100	0.000	0.200	0.200	4950	1436	614
100	0.000	0.100	0.000	4950	1436	614
100	0.000	0.000	0.000	4950	1436	614

Performance Chart





5.1 Experimental Environment

Parameter	Value to Record
Operating System	[Enter OS and version]
Processor	[Enter CPU model]
RAM	[Enter installed RAM]
Programming Language / Runtime	[Enter language and version]
Browser / IDE	[Enter browser or IDE and version]
Dataset	[Enter exact public dataset name/version]
Coordinate representation	[Enter coordinate fields / projection used]
Hybrid threshold	[Enter experimentally selected threshold]

5.2 Benchmark Result Table

Input Size n	Brute Force Time	D&C Time	Hybrid Time	BF Checks	D&C Checks	Hybrid Checks	Correct?
100	[measure]	[measure]	[measure]	[measure]	[measure]	[measure]	Yes
500	[measure]	[measure]	[measure]	[measure]	[measure]	[measure]	Yes
1,000	[measure]	[measure]	[measure]	[measure]	[measure]	[measure]	Yes
2,000	[measure]	[measure]	[measure]	[measure]	[measure]	[measure]	Yes
5,000+	[measure if feasible]	[measure]	[measure]	[measure]	[measure]	[measure]	Yes

Correctness validation should compare the minimum distance returned by all three algorithms. The point identifiers should also be compared where ties are not present. If multiple pairs have exactly the same minimum distance, the validation rule should accept any pair whose distance equals the global minimum within the selected numeric tolerance.

5.3 Memory Utilization

Memory should be measured using the same method for each algorithm. The report should distinguish input storage from auxiliary working memory where possible. Divide-and-Conquer may require additional arrays or recursion stack space, while Brute Force has very small auxiliary requirements. The Hybrid method inherits the recursive structure but can reduce practical overhead by terminating recursion earlier.

5.4 Graphs to Include

7. Input size vs execution time for all three algorithms.
8. Input size vs number of distance computations.
9. Input size vs memory utilization.
10. Hybrid threshold vs average execution time.
11. Speedup of Hybrid relative to Brute Force and Divide and Conquer.

The interactive HTML application can be used to generate immediate visual evidence during demonstration. For the final report, the measured benchmark graphs should be exported or recreated from the final experimental dataset.

6. Analysis and Engineering Decision

The theoretical analysis predicts a major scalability difference between Brute Force and Divide and Conquer. Brute Force grows quadratically because every new point can create comparisons with almost every previous point. Divide and Conquer reduces the asymptotic growth to $\Theta(n \log n)$ when the combine step is implemented efficiently. The Hybrid algorithm retains this large-input asymptotic behavior while attempting to reduce the constant overhead of recursion for small subproblems.

Experimental results should be interpreted together with the theory. For small datasets, Brute Force may be faster because it has almost no algorithmic setup cost. As the input size increases, the number of Brute-Force comparisons rises rapidly. The Hybrid algorithm is expected to follow Divide-and-Conquer behavior on large datasets while avoiding unnecessary recursive overhead near the leaves of the recursion tree.

6.1 Advantages and Limitations

Approach	Advantages	Limitations
Brute Force	Simple, transparent, easy to validate, strong baseline.	$\Theta(n^2)$; poor scalability for large inputs.
Divide and Conquer	Strong asymptotic scalability; reduces unnecessary comparisons.	More complex; recursion and ordering overhead.
Hybrid	Combines scalability with low-overhead small-case processing; adaptive threshold.	Threshold is implementation- and environment-dependent; more design complexity.

6.2 Engineering Decision

The recommended engineering decision is to use the Hybrid algorithm for large-scale infrastructure data, provided that the selected threshold is experimentally validated on the target environment. Brute Force remains valuable as a correctness oracle and as the small-input base case. Divide and Conquer provides the scalable framework. The Hybrid design is therefore preferable when the goal is to achieve scalable performance without carrying recursive overhead into very small subproblems.

The final submission should support this decision quantitatively using the completed benchmark table and graphs. The selected threshold should be stated explicitly, and the report should show that it produced the lowest or most consistently competitive runtime among tested threshold values while maintaining correctness.

7. Broader Considerations

The developed solution contributes to efficient computational resource utilization by reducing unnecessary distance computations. For large geospatial datasets, avoiding quadratic growth is important because CPU time and memory movement can become significant operational costs.

The project also demonstrates how algorithmic optimization can support infrastructure planning. A closest-pair service could be used as one component of a larger spatial-analysis system to flag potentially redundant facilities, identify locations requiring review, or support coverage optimization. The approach is practical because the Hybrid algorithm does not discard the simple Brute-Force method; instead, it uses it strategically where it is most appropriate.

The work aligns with SDG 9 – Industry, Innovation and Infrastructure because efficient computational methods can support intelligent infrastructure analysis and more scalable digital systems. Although the algorithm itself does not directly construct infrastructure, it provides a computational building block for data-driven planning systems.

8. Conclusion

This assignment addressed the Closest-Pair-of-Points problem in the context of large-scale geospatial infrastructure data. Three algorithmic approaches were developed and compared: Brute Force, Divide and Conquer, and a Hybrid approach. Brute Force provides a reliable quadratic baseline and correctness reference. Divide and Conquer improves theoretical scalability through recursive partitioning and strip processing. The Hybrid algorithm combines the two by switching to Brute Force when the recursive subproblem becomes sufficiently small.

The key development decision is the experimental determination of the switching threshold. Because the optimal threshold depends on implementation and execution environment, it should be selected from repeated benchmark results rather than assumed. The final report must insert the measured threshold, timings, distance-computation counts and memory measurements obtained from the selected real-world dataset.

The principal limitation is that planar Euclidean distance is a simplification for raw latitude/longitude coordinates over large geographic regions. Future versions can use projected coordinates or geodesic distance, parallel processing, spatial indexing, or more advanced memory-efficient implementations. Nevertheless, the Hybrid strategy demonstrates how theoretical algorithm analysis can be converted into a practical performance optimization.

9. Student Reflection

9.1 What did you learn beyond directly implementing classroom algorithms?

The main learning outcome was understanding that algorithm design is not limited to selecting the algorithm with the best asymptotic complexity. Practical performance also depends on constants, recursion overhead, data organization, sorting, memory behavior and the execution environment. Developing the Hybrid algorithm demonstrated how two individually valid techniques can be combined to obtain a more practical implementation.

9.2 What challenges were encountered while determining the threshold?

The major challenge is that a threshold that performs well for one input size or machine may not be optimal for another. A single benchmark can also be affected by background processes, browser activity or runtime warm-up. The appropriate response is to use identical inputs, repeated runs, consistent measurement procedures and correctness validation. The threshold should be selected from a group of measurements rather than from one observation.

9.3 How would you improve the algorithm with additional resources?

With additional time and computational resources, the implementation could maintain points sorted by y-coordinate throughout the recursion instead of repeatedly sorting strip data. This would make the theoretical O(n log n) behavior more directly reflected in the implementation. Further improvements could include parallel recursive processing, spatial indexing for repeated queries, efficient memory reuse, and geographic distance functions appropriate for the Earth's surface.

9.4 How can the solution contribute to SDG 9?

The solution contributes to SDG 9 by demonstrating a computationally efficient method for processing infrastructure-related spatial data. Reducing unnecessary computations improves scalability and can make data-analysis services more responsive and resource-efficient. Such algorithmic improvements are relevant to digital infrastructure systems that must process growing volumes of geographic information.

10. References

The final submission should use a consistent citation style. The following sources are suitable categories for the completed report:

E. OpenFlights. Public airport and airline datasets. Record the exact dataset URL and access date used in the experiment.
F. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms. MIT Press. Use the edition consulted for closest-pair, divide-and-conquer and asymptotic analysis.
G. A standard Design and Analysis of Algorithms textbook or course material consulted for Brute Force, Divide and Conquer, asymptotic notation and the Master Theorem.
H. Documentation for the programming language/runtime and visualization libraries used for the benchmark.
I. The standalone GeoAlgoLab HTML demonstrator developed for this assignment.
J. AI-assisted development tools, if used, should be disclosed according to institutional requirements.

K. Assignment Problem / Challenge

Modern infrastructure-planning applications process large volumes of **geospatial coordinate data**, making computationally efficient algorithms essential for identifying spatial relationships among facilities. Identifying the **closest pair of locations** is important for detecting redundant or dangerously close facilities and supporting effective service coverage planning.

Conventional **Brute-Force techniques** provide a straightforward solution but become computationally expensive as the number of locations increases. **Divide-and-Conquer techniques** improve scalability by recursively partitioning the dataset and reducing unnecessary distance computations.

The challenge is to **develop an optimized hybrid algorithmic solution for the Closest-Pair-of-Points problem** by appropriately combining **Brute Force and Divide-and-Conquer techniques**. The developed solution must determine an effective switching threshold between the two approaches and be supported by **theoretical complexity analysis and experimental evaluation using a real-world geospatial dataset**.

L. Problem Statement

An infrastructure-planning agency maintains a large, publicly available geospatial dataset of facility locations—for example, the OpenFlights airports dataset or a cell-tower/weather-station dataset containing coordinate records. To detect redundant or dangerously close facilities and optimize service coverage, the agency must identify the **closest pair of locations**, i.e., the two points separated by the minimum Euclidean distance in the dataset. Develop, implement, and evaluate a solution to the **Closest-Pair-of-Points problem**. Implement **(i) a Brute-Force algorithm** that evaluates all $n(n-1)/2$pairs with $O\left(n^2\right)$complexity and **(ii) a Divide-and-Conquer algorithm** that recursively partitions the point set about the median x-coordinate and evaluates points across the dividing strip with $O(n \log log\, n)$complexity. Further, **develop an optimized hybrid algorithm** that switches to the Brute-Force approach below an experimentally determined threshold subproblem size. Analyze the theoretical efficiency of each approach using **Big-O, Ω, and Θ notations**, and solve the recurrence $T(n) = 2T(n/2) + O(n)$using the **Master Theorem**. Experimentally evaluate all three algorithms on the selected dataset using increasing input sizes, **for example,** 10^3**,** 5×10^3**,** 10^4**, and higher input sizes wherever computationally feasible**, measuring **execution time, number of distance computations/comparisons, memory utilization, and scalability**. Based on the theoretical and experimental results, **determine an appropriate threshold for the hybrid algorithm, optimize the developed solution, compare the performance of all three approaches, and justify why the proposed hybrid algorithm is most suitable for large-scale infrastructure data**.

Expected Development

The work should **not merely implement the existing Brute-Force and Divide-and-Conquer algorithms**. The solution should demonstrate algorithmic development through:

- **Hybridization of Brute Force and Divide-and-Conquer techniques** for the Closest-Pair-of-Points problem.
- **Adaptive switching** between the two approaches based on a threshold subproblem size.
- **Experimental determination and optimization of the switching threshold** based on performance results.
- **Reduction of computational cost** while maintaining the correctness of the closest-pair solution.
- **Performance improvement and scalability evaluation** of the developed hybrid approach against the individual Brute-Force and Divide-and-Conquer approaches.

M. Requirements and Constraints

The following requirements and constraints shall be considered:

- Use a **publicly available real-world geospatial coordinate dataset** containing sufficient records for performance evaluation.
- Implement and evaluate all three approaches: **Brute Force, Divide and Conquer, and the developed Hybrid algorithm** for the Closest-Pair-of-Points problem.
- Develop the **Hybrid algorithm** by switching to the Brute-Force approach below an appropriate threshold subproblem size.
- Determine and justify the **switching threshold experimentally** based on performance results.
- Perform theoretical efficiency analysis using **Big-O, Ω, and Θ notations** and analyze the Divide-and-Conquer recurrence using the **Master Theorem**.
- Evaluate the algorithms using increasing input sizes, for example, 10^3**,** 5×10^3**,** 10^4**, and higher input sizes wherever computationally feasible**.
- Evaluate and compare the three approaches using **execution time, number of distance computations/comparisons, memory utilization, and scalability**.
- Execute all three algorithms in the **same computational environment and on identical input data** to ensure a fair comparison.
- Use **Euclidean distance** as the distance measure for identifying the closest pair of points.
- Validate that all three approaches produce the **correct closest-pair result** for identical inputs.
- Higher input sizes shall be considered only where computationally feasible, taking into account the $O(n^2)$**computational cost of the Brute-Force approach**.
- Clearly document the **dataset source, experimental environment, threshold value, assumptions, limitations, and implementation details**.

N. Student Work

1. Problem Understanding and Formulation

- Describe the **Closest-Pair-of-Points problem** in the context of real-world geospatial infrastructure planning.
- Obtain an appropriate **publicly available real-world coordinate dataset**.
- Describe the **dataset, attributes, number of records, and source**.
- Define the expected **input and output** of the problem.
- Explain the use of **Euclidean distance** for identifying the closest pair of points.
- Identify the computational challenges associated with **increasing input size**, particularly for the Brute-Force approach.
- State the relevant **assumptions, requirements, and constraints**.

2. Application of Course Knowledge

Identify and apply the relevant concepts:
CO1 – Algorithm Analysis

- Fundamentals of algorithmic problem solving
- Framework for algorithm efficiency analysis
- **Big-O, Ω, and Θ** notations
- Analysis of recursive and non-recursive algorithms
- **Master Theorem** for analyzing the Divide-and-Conquer recurrence

CO2 – Brute Force

- Apply the **Brute-Force Closest-Pair algorithm** by evaluating all $n(n-1)/2$possible pairs.
- Analyze its computational efficiency and scalability.

CO3 – Divide and Conquer

- Apply the **Divide-and-Conquer Closest-Pair algorithm** using recursive partitioning about the median x-coordinate and evaluation across the dividing strip.
- Analyze its recursive structure and computational efficiency.

Explain how concepts from **CO1, CO2, and CO3** contribute to the development of the proposed **Hybrid algorithm**.

3. Solution / Design / Methodology

Design and develop an optimized **Hybrid Closest-Pair algorithm** by appropriately integrating Brute Force and Divide-and-Conquer techniques.

The methodology should follow:

Input Dataset → Data Preparation → Brute-Force Implementation → Divide-and-Conquer Implementation → Hybrid Algorithm Development → Threshold Determination → Complexity Analysis → Experimental Evaluation → Optimization → Performance Comparison → Final Decision

Provide:

- Brute-Force algorithm/pseudocode
- Divide-and-Conquer algorithm/pseudocode
- Developed Hybrid algorithm/pseudocode
- Flowchart/design of the proposed Hybrid approach
- Explanation of the **hybrid switching strategy**
- Method used to determine the **threshold value**
- Mathematical complexity analysis
- Justification of algorithmic design and optimization decisions

4. Use of Modern Tools

Use appropriate tools wherever relevant:

- Python / C / C++ / Java
- Jupyter Notebook / Google Colab / suitable IDE
- Appropriate data-processing, performance-measurement, and visualization libraries

Provide evidence of the **implementation, source code, dataset source, experimental setup, execution outputs, tables, and graphs**.

5. Results and Validation

Experimentally evaluate all three approaches—**Brute Force, Divide and Conquer, and the developed Hybrid algorithm**—using increasing input sizes.

The evaluation should include:

- Execution time
- Number of distance computations/comparisons
- Memory utilization
- Input size
- Scalability
- Theoretical time complexity
- Correctness of the closest-pair result
- Effect of the selected **hybrid threshold**

Present the experimental results using suitable **tables and graphs**.

Compare the performance of the **developed Hybrid algorithm** against the Brute-Force and Divide-and-Conquer approaches using identical input data and the same computational environment.

6. Analysis and Engineering Decision

Interpret the theoretical and experimental results and:

- Compare **Brute Force, Divide and Conquer, and the developed Hybrid algorithm**.
- Analyze the relationship between **theoretical and experimental performance**.
- Evaluate the effect of increasing input size on each approach.
- Analyze the impact of the **switching threshold** on Hybrid algorithm performance.
- Identify the advantages and limitations of each approach.
- Discuss computational and scalability trade-offs.
- Determine the conditions under which each approach performs effectively.
- Justify the **selected threshold value** for the Hybrid algorithm.
- Determine and justify the most suitable approach for **large-scale geospatial infrastructure data**.
- Provide **quantitative experimental evidence** supporting the final engineering decision.

7. Broader Considerations

Discuss how the developed solution contributes to:

- Efficient utilization of computational resources
- Reduction in unnecessary distance computations
- Improved scalability for large geospatial datasets

- Efficient infrastructure planning and analysis
- Practical applicability to real-world systems
- Sustainable and computationally efficient solution development

Relate the developed solution to **SDG 9 – Industry, Innovation and Infrastructure**, particularly its potential contribution to efficient and intelligent infrastructure planning.

8. Conclusion

Summarize:

- Closest-Pair-of-Points problem addressed
- Brute-Force and Divide-and-Conquer approaches implemented
- Developed Hybrid algorithm
- Experimentally determined switching threshold
- Major experimental findings
- Best-performing approach
- Complexity and scalability improvements achieved
- Limitations
- Possible future optimization

9. Student Reflection

Answer individually:

1. What did you learn from developing and optimizing the Hybrid Closest-Pair algorithm beyond implementing the algorithms directly taught in the classroom?
2. What challenges did you encounter while determining the switching threshold and evaluating performance across different input sizes, and how did you address them?
3. If additional computational resources or time were available, how would you further improve the developed algorithm and why?
4. How can the developed solution contribute to SDG 9 – Industry, Innovation and Infrastructure?

10. References

Cite:

- Dataset source
- Algorithms/research literature consulted
- Software/tools/libraries used
- Other external resources
- AI-assisted tools, if used
- Use an appropriate and consistent citation format.

O.Assessment Rubric

Criteria	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Algorithmic Formulation	**10**	Clearly identifies the problem, requirements, inputs, outputs, constraints and computational challenges and formulates them appropriately for solution development.	Problem and major requirements are correctly formulated with minor gaps.	Basic formulation is provided, but requirements or constraints are partially identified.	Problem is poorly understood or inadequately formulated.
Algorithmic Solution Design	**15**	Designs a clear, logical and feasible solution strategy with appropriate algorithmic techniques, pseudocode/flowchart and well-justified design decisions.	Appropriate solution design with minor gaps in logic, representation or justification.	Basic solution design is presented but lacks completeness or clear justification.	Solution design is inappropriate, incomplete or unclear.
Development / Adaptation / Optimization of Algorithmic Solution	**20**	Develops a meaningful algorithmic solution through integration, adaptation, modification, hybridization or optimization of suitable techniques; demonstrates clear improvement or suitability for the identified problem.	Develops an appropriate solution with meaningful adaptation or improvement, with minor limitations.	Solution demonstrates limited adaptation or development beyond standard implementation.	Merely reproduces an existing algorithm with little or no meaningful development or adaptation.
Implementation & Modern Tool Usage	**15**	Developed solution is correctly and efficiently implemented using appropriate programming/computati	Implementation is functional with minor logical or	Core implementation works but contains limitations or	Implementation is incomplete, unreliable or

		onal tools and handles relevant input conditions reliably.	efficiency issues.	incomplete functionality.	non-function al.
Efficiency, Testing & Validation	**15**	Performs appropriate complexity evaluation and systematic testing using suitable data/input sizes; validates correctness, efficiency and scalability with clear evidence.	Complexity evaluation and testing are mostly appropriate with minor gaps.	Basic testing and efficiency evaluation are provided but lack sufficient validation.	Testing, complexity evaluation or validation is inadequate or substantially incorrect.
Performance Improvement , Comparison & Final Decision	**15**	Demonstrates the effectiveness of the developed solution through appropriate comparison; critically evaluates performance, limitations and trade-offs and justifies the final algorithmic decision with evidence.	Good performance comparison and justification with minor limitations.	Basic comparison is provided, but improvement or final justification lacks depth.	Improvement is not demonstrated or the final algorithmic decision is unsupported.
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	**10**	Thoughtful justification of algorithmic design and development decisions; clear connection to relevant SDG(s); specific reflection on computational challenges, performance improvements, trade-offs, and learning gained.	Appropriate justification of algorithmic decisions; SDG relevance, challenges, improvement s, and learning are discussed but lack depth.	Reflection is present but generic; limited justification of algorithmic decisions, improvements , SDG relevance, or learning outcomes.	No genuine reflection is provided, or the reflection lacks meaningful connection to algorithm development, SDG relevance, challenges, or learning outcomes.
Total	**100**				

P. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem Understanding & Algorithmic Formulation	CO1	PO1, PO2	L4	**10**
Algorithmic Solution Design	CO2, CO3	PO2, PO3	L4, L6	**15**
Development / Adaptation / Optimization of Algorithmic Solution	CO2, CO3	PO2, PO3	L5, L6	**20**
Implementation & Modern Tool Usage	CO2, CO3	PO3, PO5	L3, L6	**15**
Efficiency, Testing & Validation	CO1, CO2, CO3	PO2, PO4	L4, L5	**15**
Performance Improvement, Comparison & Final Decision	CO1, CO2, CO3	PO2, PO4	L4, L5	**15**
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	CO1, CO2, CO3	PO7, PO10, PO12	L4, L5	**10**

Q. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	≥ 70%
Level 2	60–69%
Level 1	50–59%
Level 0	< 50%

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action: